DIGT 3107 - Room/Hotel Booking System

REPOSITORY LINK

Phase 4

Student#:	**219941855**	**220329405**	**220486114**
Sur (Family) Name:	**Carter**	**Chawda**	**Dhawan**
Given Name:	**Toby**	**Gira**	**Komal**
Section:	**A**	**A**	**A**

Overview of System

The hotel booking system is a management system for a small hotel. The system is built in SQL, with five tables:

- A guests table for guest information including their name, contact information, and if they're a staff member for discount purposes.
- A rooms table for tracking individual rooms, including room id, room number, and type.
- A room types table for tracking the type, capacity, and nightly price of each category.
- A payment methods table to track payment methods.
- A central bookings table that links to rooms, guests, and payment methods to track individual bookings and all associated information.

This system allows for the booking of the rooms in the hotel, associating bookings with guests and payment methods, and tracking financials from bookings, and summaries by time period. The database can be queried using SQL to input new records, and query existing ones to gather information, and additionally can be interacted with via a text-based interface.

Functional Description

The system allows for the creation, deletion, and tracking of rooms, bookings, and guests. This includes listing available rooms for specific dates and ranges, bookings made by a specific guest, most booked room types, the total number of bookings and revenue for a year with discounts calculated, and the revenue per room type for a given month, also with discounts calculated. Additionally, booking cancellations can be tracked. We demonstrate many of these functional requirements in our complex queries file in phase 3

Additionally, a text-based menu interface connected to the database through JDBC allows for easy access to seven common flows without requiring direct SQL:

1. View All Rooms:
This showcases all of the rooms the hotel offers, and all relevant information for each room, including Room ID, Room #, Type, Capacity, and Price/Night.

2. Check Room Availability:
This flow allows for the user to enter a specific date, and any room not booked on that night will be shown, along with all relevant information including Room ID, Room #, Type, Capacity, and Price/Night.

3. Add New Booking:
This is one of the core flows, and allows for the user to create a new booking for either an existing or new guest. If an exiting guest is selected, the user will be prompted for their name, room ID, payment method ID, check-in date, and check-out date. If a new guest is entered, then the user will be prompted to provide their email address and if they're a staff member or not (for discount purposes). The system then outputs the new guest ID and

continues the booking flow. If an invalid date is inputted then the user will be informed and asked to begin the flow again.

4. View All Bookings:
This flow lists the last 20 bookings created in the system, and all relevant information, including Booking ID, Guest Name, Room #, Check-in date, Check-out date, and Status (Active or Cancelled).

5. Update Booking Discount:
This flow prompts the user for a booking ID, and if the staff discount should be applied or removed. This then updates the booking in the database, and all corresponding financial summaries to include the discount amount.

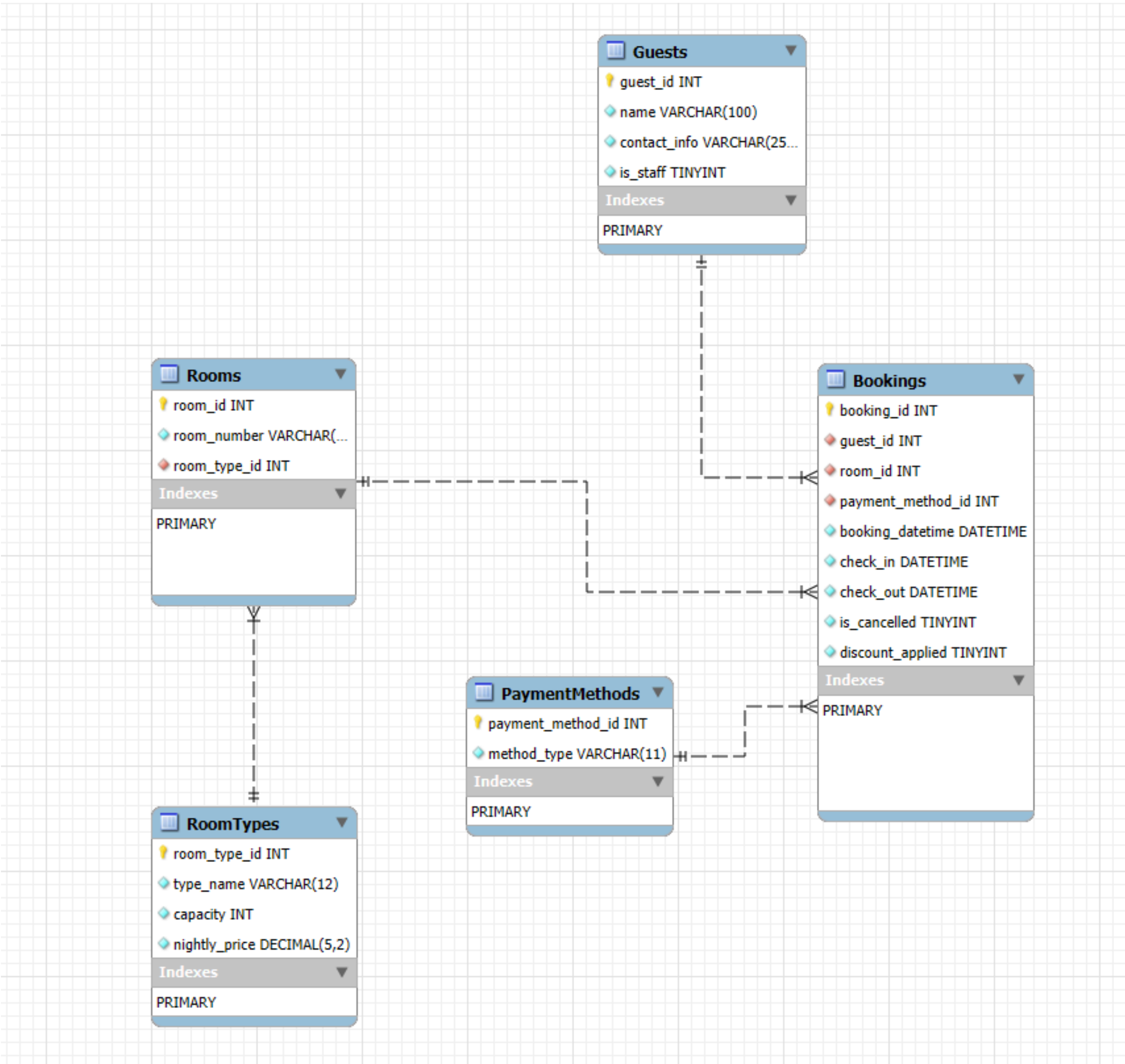
6. Cancel Booking:
This prompts the user for a booking ID, and then updates the booking in the database to have a status of “Cancelled”. It is then also no longer included in any financial summary.

7. View Weekly Revenue Report:
This shows the top 10 revenue weeks in the database, including they Year, Week, and Revenue. The revenue is calculated based on active (not cancelled) stays, the room category, and any discount applied.

Java-Database Connection

We utilized JDBC (Java Database Connectivity) to connect our Java text-based console interface to our database through localhost. It connects using the following url: “`jdbc:mysql://localhost:3306/hotel_booking?useSSL=false&serverTimezone=UTC`" and the username and password configured in the file by the user for local testing. We don’t have a password set by default as this is for local use only. We also handle any failures with the JDBC driver elegantly, including not being able to find the driver (this can happen when the path isn’t set correctly, to mitigate this we include a VSCode settings file), or a SQL exception. By using this API, we can execute queries against our database from Java, utilizing our text-based menu.

ER Diagram



Schemas

Guests Table:

	Field	Type	Null	Key	Default	Extra
▶	guest_id	int	NO	PRI	NULL	auto_increment
	name	varchar(100)	NO		NULL	
	contact_info	varchar(254)	NO		NULL	
	is_staff	tinyint(1)	NO		0	

Payment Methods Table:

	Field	Type	Null	Key	Default	Extra
▶	payment_method_id	int	NO	PRI	NULL	auto_increment
	method_type	varchar(11)	NO		NULL	

Room Types Table:

	Field	Type	Null	Key	Default	Extra
▶	room_type_id	int	NO	PRI	NULL	auto_increment
	type_name	varchar(12)	NO		NULL	
	capacity	int	NO		NULL	
	nightly_price	decimal(5,2)	NO		NULL	

Rooms Table:

	Field	Type	Null	Key	Default	Extra
▶	room_id	int	NO	PRI	NULL	auto_increment
	room_number	varchar(3)	NO	UNI	NULL	
	room_type_id	int	NO	MUL	NULL	

Bookings Table:

	Field	Type	Null	Key	Default	Extra
▶	booking_id	int	NO	PRI	NULL	auto_increment
	guest_id	int	NO	MUL	NULL	
	room_id	int	NO	MUL	NULL	
	payment_method_id	int	NO	MUL	NULL	
	booking_datetime	datetime	NO		NULL	
	check_in	datetime	NO		NULL	
	check_out	datetime	NO		NULL	
	is_cancelled	tinyint(1)	NO		0	
	discount_applied	tinyint(1)	NO		0	